

Objectives :

- Build a virtual network and verify connectivity

This lab was adapted to run entirely on laptops, without Cisco hardware, by using Linux network *namespaces*, *virtual interfaces*, and *iptables* to simulate routers, switches, and NAT behavior. Experiments can be conducted with *network connectivity*, *static NAT*, and *dynamic NAT* fully in software.

Network scenario and requirements

Background / Scenario

In this lab:

- The **entity's network** uses private IPs (192.168.10.0/24 and 192.168.20.0/24)
- The **ISP provides public IPs** (simulate 209.165.200.224/27)
 - Static: 209.165.200.225–209.165.200.241
 - Dynamic: 209.165.200.242–209.165.200.254
- A **gateway router** (simulated) connects the LAN to the “ISP” (another namespace)
- Connectivity testing is done with ping

Required Software

- Linux laptop (Ubuntu/Debian recommended)
- iproute2 (for network namespaces)
- ping (for connectivity and DNS testing)

Lab Topology (Virtual)

<pre> [PC1]-- --[Gateway Router]--[ISP Router]--(Internet) [PC2]-- </pre>	• LAN 1 ◦ PC1: 192.168.10.10/24 ◦ Gateway (LAN1): 192.168.10.1/24 • LAN 2 ◦ PC2: 192.168.20.10/24 ◦ Gateway (LAN2): 192.168.20.1/24 • WAN ◦ ISP: 209.165.200.225/27 ◦ Gateway (WAN): 209.165.200.226/27
--	---

Part 1: Build a virtual network

1. In the Linux lab, which network namespace plays the role of:

- Cisco PC
 - Cisco Gateway Router
 - Cisco ISP Router
- | |
|---|
| • Cisco PC: PC1, PC2 namespaces |
| • Cisco Gateway Router: GATEWAY namespace |
| • Cisco ISP Router: ISP namespace |

2. Fill out the following Linux mapping table

Cisco Component	Linux Equivalent
PC	namespace (PC1, PC2)
Router	Network namespace (GATEWAY, ISP)
Interface G0/0	veth interface
Interface G0/1	veth interface
Public IP block	IP address on veth3
Internet cloud	ISP namespace + loopback

3. Why namespaces is equivalent to Cisco devices?

Each network namespace has its own interfaces, its own routing table and its own firewall/NAT rules. This is equivalent to how each Cisco router operates independently.

4. Fill out the following Linux tables regarding interface and addressing mapping

Gateway interfaces	
Inside interfaces	gw1, gw2
Outside interface	veth3
Verification command	ip addr

Cisco Interface	Linux Interface
G0/0 (LAN1)	gw1
G0/0 (LAN2)	gw1
G0/1 (WAN)	veth3
ISP G0/0	isp1

5. Follow these steps for deployment and testing:

Step 1: Setup Network Namespaces

```
#Create namespaces, check ls -l /var/run/netns/1
sudo ip netns add PC1
sudo ip netns add PC2
sudo ip netns add GATEWAY
sudo ip netns add ISP
```

```
#Create veth pairs to connect namespaces
sudo ip link add veth1 type veth peer name gw1
sudo ip link add veth2 type veth peer name gw2
sudo ip link add veth3 type veth peer name isp1
```

```
#Assign veth ends to namespaces
sudo ip link set veth1 netns PC1
sudo ip link set veth2 netns PC2
sudo ip link set gw1 netns GATEWAY
sudo ip link set gw2 netns GATEWAY
sudo ip link set veth3 netns GATEWAY
sudo ip link set isp1 netns ISP
```

¹ Please note that when typing `sudo ip netns add PC1` Linux creates `/var/run/netns/PC1`, a **symlink** (or handle) pointing to a namespace object in `/proc/<pid>/ns/net`. Will be handled in Lab1.

```
# Bring up interfaces inside namespaces
#PC1
sudo ip netns exec PC1 ip addr add 192.168.10.10/24 dev veth1
sudo ip netns exec PC1 ip link set veth1 up
sudo ip netns exec PC1 ip link set lo up
#PC2
sudo ip netns exec PC2 ip addr add 192.168.20.10/24 dev veth2
sudo ip netns exec PC2 ip link set veth2 up
sudo ip netns exec PC2 ip link set lo up
#GATEWAY (LAN side)
sudo ip netns exec GATEWAY ip addr add 192.168.10.1/24 dev gw1
sudo ip netns exec GATEWAY ip addr add 192.168.20.1/24 dev gw2
sudo ip netns exec GATEWAY ip link set gw1 up
sudo ip netns exec GATEWAY ip link set gw2 up
sudo ip netns exec GATEWAY ip link set lo up
#GATEWAY (WAN side)
sudo ip netns exec GATEWAY ip addr add 209.165.200.225/27 dev veth3
sudo ip netns exec GATEWAY ip link set veth3 up
#ISP
sudo ip netns exec ISP ip addr add 209.165.200.226/27 dev isp1
sudo ip netns exec ISP ip link set isp1 up
sudo ip netns exec ISP ip link set lo up
```

```
# Enable loopback in all namespaces (already done)
for ns in PC1 PC2 GATEWAY ISP; do sudo ip netns exec $ns ip link set lo up done
```

Step 2: Routing Configuration

```
# Add routes to reach the ISP, Default gateways for PCs and GATEWAY
sudo ip netns exec PC1 ip route add default via 192.168.10.1
sudo ip netns exec PC2 ip route add default via 192.168.20.1
sudo ip netns exec GATEWAY ip route add default via 209.165.200.226
# Enable IP forwarding on GATEWAY
sudo ip netns exec GATEWAY sysctl -w net.ipv4.ip_forward=1
```

Step 3: Automated Bash script of Network configuration

Prepare a Bash script that automatically creates, configures, and cleans up a complete Linux network laboratory using network namespaces. The objective is to automate network deployment for future reuse and to avoid manual configuration. The script must include:

- Clean-up mechanism
- Namespace creation
- Virtual link creation
- IP addressing
- Interface and loopback activation
- Routing configuration and gateway behavior

Step 4: Automated Bash script of full cleanup and reset of the network configuration

Create a Bash script to fully clean and reset the created network environment. This script is intended to completely remove all virtual networking components created during the lab so that the entire setup can be rebuilt from scratch without conflicts.

Part 2: Connectivity testing

Once you've set up your network namespaces, veth links, IPs, and routes, there are several Linux commands you can use to inspect and verify the configuration.

Scenario 1: Namespace existence

```
# Verify that all namespaces were created correctly.  
ip netns list  
ISP (id: 3)  
GATEWAY (id: 2)  
PC2 (id: 1)  
PC1 (id: 0)
```

Scenario 2: Interface placement

```
# Ensure veth interfaces are in the correct namespaces.  
sudo ip netns exec PC1 ip link  
sudo ip netns exec PC2 ip link  
sudo ip netns exec GATEWAY ip link  
sudo ip netns exec ISP ip link  
PC1 → veth1; PC2 → veth2; GATEWAY → gw1, gw2, veth3; ISP → isp1
```

Scenario 3: IP configuration

```
# Confirm IP addresses and subnets.  
sudo ip netns exec PC1 ip addr  
sudo ip netns exec PC2 ip addr  
sudo ip netns exec GATEWAY ip addr  
sudo ip netns exec ISP ip addr  
PC1: 192.168.10.10/24  
PC2: 192.168.20.10/24  
GATEWAY LAN: .1/24  
GATEWAY WAN: 209.165.200.226/27  
ISP: 209.165.200.225/27
```

Scenario 4: Local Connectivity Tests

```
# PC ↔ Gateway  
sudo ip netns exec PC1 ping 192.168.20.10  
sudo ip netns exec PC2 ping 192.168.10.10
```

Scenario 5: Inter-LAN Routing Test

```
# PC1 ↔ PC2 to test if IP forwarding works, No NAT involved yet  
sudo ip netns exec PC1 ping 192.168.10.1  
sudo ip netns exec PC2 ping 192.168.20.1
```

Scenario 6: Gateway ↔ ISP Link Test

```
# Gateway → ISP, validate WAN link correctness and Proper addressing  
sudo ip netns exec GATEWAY ping 209.165.200.225  
sudo ip netns exec ISP ping 209.165.200.226
```

Scenario 7: End-to-End Routing Test (Before NAT)

```
# PC → ISP interface  
sudo ip netns exec PC1 ping 209.165.200.225
```

The output shows that the ping may fail if ISP has no return route and the need for NAT or ISP static route. Please note that this command is not just a ping, it is an end-to-end routing correctness test. It simultaneously tests the following

- Default route on PC1
- Forwarding on GATEWAY
- Routing symmetry (return path)
- Need for NAT or ISP static routes

To better understand the issue here is a step-by-step packet traces.

From the PC1, routing table: default via 192.168.10.1, packet goes to GATEWAY

- Source IP: 192.168.10.10
 - Destination IP: 209.165.200.225
- At the GATEWAY, the routing table: default via 209.165.200.225, packet forwarded to ISP.
- At the ISP, the routing table: 209.165.200.224/27 dev isp1ISP does NOT know how to reach 192.168.10.0/24. It Breaks at the return Path.

To reply to the ping, ISP must send: DST = 192.168.10.10. But ISP has no default route and no static route to 192.168.10.0/24, echo reply is dropped. This failure proves one fundamental networking rule, routing must work in BOTH directions.

In order to figure out that, test the following scenarios:

Scenario 1: Prove packet reaches ISP

```
# Run on the ISP
sudo ip netns exec ISP tcpdump -i isp1 icmp
# Ping again from PC1.
sudo ip netns exec PC1 ping 209.165.200.225
```

You will see ICMP Echo Request, but NOT see Echo Reply. This confirms one-way connectivity. To fix the failure test the following:

Scenario 2: Prove packet reaches ISP

```
# Add route
sudo ip netns exec ISP ip route add 192.168.10.0/24 via 209.165.200.226
sudo ip netns exec ISP ip route add 192.168.20.0/24 via 209.165.200.226
# Ping again from PC1
sudo ip netns exec PC1 ping 209.165.200.225
# Ping again from PC2
sudo ip netns exec PC2 ping 209.165.200.225
```

Scenario 3: repeat scenario 1

```
# Run on the ISP
sudo ip netns exec ISP tcpdump -i isp1 icmp
# Ping again from PC1.
sudo ip netns exec PC1 ping 209.165.200.225
```

Modify your script to take the abovementioned modifications. Test the connectivity again (*PC* → *ISP*).

Additional items

Q1: What network namespaces were created by the script?

- PC1, PC2, GATEWAY, ISP
- PC1: First LAN host
- PC2: Second LAN host
- GATEWAY: Router connecting the two LANs and the ISP
- ISP: Represents the Internet provider

Q2: Which interfaces does each namespace have?

- PC1: veth1
- PC2: veth2
- GATEWAY: gw1, gw2, veth3
- ISP: isp1

Q3: What IP addresses are assigned to veth1, veth2, gw1, gw2, veth3, isp1 ?

Q4: Which device plays the role of a router?

Namespace	Interface	IP Address
PC1	veth1	192.168.10.10/24
PC2	veth2	192.168.20.10/24
GATEWAY	gw1	192.168.10.1/24
GATEWAY	gw2	192.168.20.1/24
GATEWAY	veth3	209.165.200.226/27
ISP	isp1	209.165.200.225/27

GATEWAY namespace acts as a router because it connects multiple networks (LAN1, LAN2, WAN) and forwards packets between them.

Q5: Why can PC1 ping the Gateway but not the ISP directly?

- PC1 is in 192.168.10.0/24; Gateway is its default route → reachable
- ISP is on 209.165.200.224/27 → not directly reachable because IP forwarding is needed and the packet must be routed, and NAT isn't configured yet.

Q6: What default route is configured on PC1 and PC2?

- PC1: default via 192.168.10.1
- PC2: default via 192.168.20.1
- This tells each host to send all unknown-destination packets to the Gateway.

Q7: What is the default route on the Gateway? Why?

- default via 209.165.200.225
- This points to the ISP for any packets destined outside the LANs. Without it, the Gateway wouldn't forward traffic to the Internet.

Q8: Explain what happens when PC1 sends a packet to 209.165.200.225.

- Packet leaves PC1 → veth1 → Gateway (gw1)
- Gateway checks routing table → sends packet via veth3 to ISP (isp1)
- Without NAT, PC1 cannot receive a reply because ISP sees private IP 192.168.10.10 (non-routable)

Q9: What is the effect of `sysctl -w net.ipv4.ip_forward=1`?

- Enables the Gateway to forward IP packets between interfaces → turns it into a router.

Q10: What would happen if IP forwarding was disabled?

- Gateway would not forward packets between LANs or to ISP → PC1 cannot reach PC2 or ISP.

Q11: Does NAT exist at this stage?

- No. Only IP forwarding is enabled; packets from LANs still have private IPs.

Q12: Does NAT exist at this stage?

- No. ISP cannot route packets from private IP 192.168.10.10. NAT is required.

Q13: What problem will NAT solve?

- Translates private LAN IPs → public IPs, allowing Internet communication.

Q14: What you can ping without NAT, test the following and comment?

Within the same LAN / namespace subnet:

- `sudo ip netns exec PC1 ping -c 4 192.168.10.1` # Gateway interface in PC1 subnet
- `sudo ip netns exec PC2 ping -c 4 192.168.20.1` # Gateway interface in PC2 subnet

Across subnets through the Gateway:

- `sudo ip netns exec PC1 ping -c 4 192.168.20.10`
- `sudo ip netns exec PC2 ping -c 4 192.168.10.10`

Works because Gateway forwards packets between PC1 and PC2 networks (private-to-private).

Public IPs / outside world:

- `sudo ip netns exec PC1 ping -c 4 209.165.200.225` # ISP
- `sudo ip netns exec PC2 ping -c 4 209.165.200.225`

Fails, as stated above, because ISP sees source IP 192.168.x.x, which is private & non-routable.

Deliverable (To be uploaded on the Platform)

In this lab, you must design and configure a Linux-based gateway using network namespaces to provide NAT connectivity for two LAN hosts, verify address translation and forwarding using iptables and packet captures, and submit configuration evidence and explanations demonstrating correct network operation. The document (word/pdf, 10 pages Max) must include the following:

- Network topology description
 - Diagram or clear explanation of the namespaces (PC1, PC2, GATEWAY, ISP)
 - IP addressing plan for each interface (LAN and WAN)
- Namespace and interface configuration
 - Commands used to create network namespaces
 - Veth pairs configuration and namespace assignment
 - Interface activation and routing configuration
- Gateway configuration
 - IP forwarding activation on the GATEWAY namespace
 - Explanation of the gateway's role in the network
- Verification and testing
 - Ping tests from PC1 and PC2 to external addresses
 - Routing table verification
 - iptables rule counters (iptables -L -v -n)
 - Packet capture evidence (tcpdump) showing NAT translation
- Conclusion
 - Summary of achieved objectives
 - Short reflection on NAT and namespace-based networking

A single compressed file named in the format “lastname.firstname-Lab0.zip”, must be uploaded on the platform (see deadlines) and must include:

- The main report document (Word or PDF)
- All developed Bash scripts

Appendix

Network Namespace Basics (Build the virtual network)

```
ip netns list # list namespaces
ip netns add <namespace> #Create a host
ip netns del <namespace> #Delete a host
ip netns exec <namespace> <command> # Execute commands inside a virtual node
```

Virtual Links (connect the network)

```
# Simulate cables between devices
ip link add veth1 type veth peer name veth2
ip link set veth1 netns PC1
ip link set veth2 netns GATEWAY
```

Virtual Links (connect the network)

```
# Assign IP addresses
ip addr add 192.168.10.10/24 dev veth1 # Give hosts IP addresses
ip addr show # Activate network interfaces
```

Routing (make packets move)

```
# ip route show
ip addr add 192.168.10.10/24 dev veth1 # Give hosts IP addresses
ip addr show # Activate network interfaces
# Add default route
ip route add default via 192.168.10.1 # Control where packets go
```

Connectivity Verification

```
# Verify Layer-3 connectivity
ping 192.168.10.1
ping -c 3 192.168.20.10
```

Inspection & Debugging Commands

```
# Interface state
ip addr
ip link
# Route
ip route
# Packet capture, see what is really happening
tcpdump -i veth1
tcpdump -i any
```

Cleanup & Reset

```
# Reset lab safely, avoid configuration pollution
ip addr flush dev veth1
ip route flush table main
ip link delete veth1
ip netns del PC1
```